

Tuyển tập Bài tập: Các Thuật toán Xử lý Đoạn & Khoảng trên Đường thẳng (1D Interval Processing)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Biên soạn: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Danh mục Bài tập Tuyển chọn

STT	Tên bài toán	Nguồn	Cấp độ	Kỹ thuật trọng tâm	Độ phức tạp
1	Merge Intervals	LeetCode 56 / VNOJ	● Cơ bản	Sắp xếp & Gộp các đoạn giao nhau	$O(n \log n)$
2	Restaurant Customers	CSES 1619	● Cơ bản	Dòng quét 1D (Sweep-line) đếm số tầng phủ cực đại	$O(n \log n)$
3	Non-overlapping Intervals	LeetCode 435	● Cơ bản	Interval Scheduling (Tham lam loại bỏ ít nhất)	$O(n \log n)$
4	Nested Ranges Check	CSES 2168	● Cơ bản	Sắp xếp đa khóa ($L \uparrow, R \downarrow$) & Min/Max quét	$O(n \log n)$
5	Many Segments	AtCoder ABC 207C	● Cơ bản	Chuẩn hóa 4 loại biên: đóng, mở, nửa mở	$O(n^2)$
6	Nested Ranges Count	CSES 2169	● Trung cấp	Nén tọa độ & Cây Fenwick (BIT) đếm số đoạn chứa	$O(n \log n)$
7	Meeting Rooms II	LeetCode 253	● Trung cấp	Min-Heap quản lý thời điểm kết thúc phòng họp	$O(n \log n)$
8	Maximum Profit in Job Scheduling	LeetCode 1235	● Trung cấp	Weighted Interval Scheduling (DP + Binary Search)	$O(n \log n)$
9	Movie Festival II	CSES 1632	● Trung cấp	Tham lam xếp K phòng song song với <code>multiset</code>	$O(n \log k)$
10	Two Platforms	Codeforces 1409E	● Trung cấp	Phủ 2 đoạn dài K + Mảng hậu tố cực đại	$O(n \log n)$

STT	Tên bài toán	Nguồn	Cấp độ	Kỹ thuật trọng tâm	Độ phức tạp
11	Video Stitching	LeetCode 1024	● Trung cấp	Minimum Interval Cover (Phủ kín đoạn ít đoạn nhất)	$O(n \log n)$
12	Remove Covered Intervals	LeetCode 1288	● Trung cấp	Sắp xếp ($L \uparrow, R \downarrow$) lọc đoạn bị bao	$O(n \log n)$
13	Packing Under Range Regulations	AtCoder ABC 214E	● Nâng cao	Gán điểm vào khoảng với Min-Heap	$O(n \log n)$
14	Moving Points	Codeforces 1311F	● Nâng cao	Nén tọa độ & 2 cây BIT tính tổng khoảng cách 1D	$O(n \log n)$
15	Maximum White Subtree	Codeforces 1324F	● Nâng cao	Rerooting DP kết hợp nén khoảng cách trên cây	$O(n)$
16	Interval Selection	AtCoder ARC	● Nâng cao	Dòng quét kết hợp Segment Tree Lazy Propagation	$O(n \log n)$

● Phần 1: Cấp độ Cơ bản & Khởi động (Warm-up)

Bài 1: Merge Intervals (LeetCode 56 / VNOJ)

- **Đề bài:** Cho một mảng các đoạn thẳng `intervals`, trong đó $\text{intervals}[i] = [L_i, R_i]$. Hãy gộp tất cả các đoạn thẳng có phần giao nhau và trả về mảng các đoạn thẳng rời rạc phủ đúng tập điểm ban đầu.
- **Ràng buộc:** $1 \leq n \leq 10^5, 0 \leq L_i \leq R_i \leq 10^9$.
- **Gợi ý:**
 - *Hint 1:* Sắp xếp các đoạn theo đầu mút trái L_i tăng dần.
 - *Hint 2:* Duy trì đoạn hiện tại $[cur_L, cur_R]$. Với mỗi đoạn tiếp theo, nếu $L_{next} \leq cur_R$, ta gộp bằng cách cập nhật $cur_R = \max(cur_R, R_{next})$.
 - *Hint 3:* Nếu $L_{next} > cur_R$, đoạn hiện tại đã hoàn chỉnh; lưu vào kết quả và bắt đầu một đoạn mới.

Bài 2: Restaurant Customers (CSES 1619)

- **Đề bài:** Có n khách hàng ghé thăm một nhà hàng. Khách thứ i đến lúc a_i và rời đi lúc b_i (khoảng thời gian có mặt là $[a_i, b_i]$). Hãy tìm số lượng khách hàng tối đa có mặt trong nhà hàng tại cùng một thời điểm.
- **Ràng buộc:** $1 \leq n \leq 2 \cdot 10^5, 1 \leq a_i < b_i \leq 10^9$.
- **Gợi ý:**
 - *Hint 1:* Biến đổi mỗi khách thành 2 sự kiện: $(a_i, +1)$ và $(b_i, -1)$.
 - *Hint 2:* Sắp xếp $2n$ sự kiện theo tọa độ thời gian tăng dần.
 - *Hint 3:* Vì khách rời đi tại b_i thì không còn tính là ở nhà hàng tại thời điểm b_i , nếu sự kiện trùng giờ, sự kiện -1 phải được xử lý trước $+1$. Duyệt từ trái qua phải, cộng dồn và cập nhật giá trị cực đại.

Bài 3: Non-overlapping Intervals (LeetCode 435)

- **Đề bài:** Cho một tập n đoạn thẳng $[L_i, R_i]$. Hãy tìm số lượng đoạn thẳng **ít nhất** cần loại bỏ để tất cả các đoạn thẳng còn lại không giao nhau.
 - **Ràng buộc:** $1 \leq n \leq 10^5$, $-5 \cdot 10^4 \leq L_i < R_i \leq 5 \cdot 10^4$.
 - **Gợi ý:**
 - *Hint 1:* Loại bỏ ít đoạn nhất tương đương với việc chọn ra số lượng đoạn không giao nhau **nhều nhất**!
 - *Hint 2:* Đây chính là bài toán Interval Scheduling kinh điển. Sắp xếp các đoạn theo thời điểm kết thúc R_i tăng dần.
 - *Hint 3:* Tham lam chọn đoạn kết thúc sớm nhất không đè lên đoạn trước. Kết quả là $n -$ (số đoạn giữ lại).
-

Bài 4: Nested Ranges Check (CSES 2168)

- **Đề bài:** Cho n đoạn thẳng $[L_i, R_i]$. Với mỗi đoạn, hãy xác định:
 - Đoạn này có chứa ít nhất một đoạn khác hay không?
 - Đoạn này có bị ít nhất một đoạn khác chứa nó hay không?
 - **Ràng buộc:** $1 \leq n \leq 2 \cdot 10^5$, $1 \leq L_i < R_i \leq 10^9$.
 - **Gợi ý:**
 - *Hint 1:* Sắp xếp các đoạn theo L tăng dần; nếu L bằng nhau thì R giảm dần.
 - *Hint 2:* Sau khi sắp xếp, đoạn j nằm trong đoạn i ($j > i$) khi và chỉ khi $R_j \leq R_i$.
 - *Hint 3:* Quét từ trái sang phải duy trì `max_R` để tìm các đoạn bị chứa. Quét từ phải sang trái duy trì `min_R` để tìm các đoạn chứa đoạn khác. Độ phức tạp $O(n \log n)$.
-

Bài 5: Many Segments (AtCoder ABC 207C)

- **Đề bài:** Cho n đoạn trên trục số. Mỗi đoạn thuộc 1 trong 4 dạng:
 - $t_i = 1$: $[L_i, R_i]$ (đóng cả hai đầu)
 - $t_i = 2$: $[L_i, R_i)$ (đóng trái, mở phải)
 - $t_i = 3$: $(L_i, R_i]$ (mở trái, đóng phải)
 - $t_i = 4$: (L_i, R_i) (mở cả hai đầu)
 - Đếm số cặp (i, j) với $i < j$ mà hai đoạn có ít nhất một điểm chung.
 - **Ràng buộc:** $1 \leq n \leq 2000$, $1 \leq L_i < R_i \leq 10^9$.
 - **Gợi ý:**
 - *Hint 1:* Để đưa tất cả 4 dạng về cùng một chuẩn đoạn đóng, ta có thể nhân đôi tọa độ hoặc co/dãn biên một lượng nhỏ 0.5 (hoặc chuyển sang số thực).
 - *Hint 2:* Quy đổi sang khoảng số thực $[l_i, r_i]$:
 - Nếu mở trái $\implies l'_i = L_i + 0.1$. Nếu đóng trái $\implies l'_i = L_i$.
 - Nếu mở phải $\implies r'_i = R_i - 0.1$. Nếu đóng phải $\implies r'_i = R_i$.
 - *Hint 3:* Hai đoạn $[l_1, r_1]$ và $[l_2, r_2]$ giao nhau khi và chỉ khi $\max(l_1, l_2) \leq \min(r_1, r_2)$. Duyệt mọi cặp trong $O(n^2)$.
-

● Phần 2: Cấp độ Trung cấp (Intermediate)

Bài 6: Nested Ranges Count (CSES 2169)

- **Đề bài:** Tương tự bài CSES 2168, nhưng thay vì chỉ trả lời Có/Không, bạn cần đếm chính xác:
- Đoạn thứ i chứa bao nhiêu đoạn khác?
- Đoạn thứ i bị bao nhiêu đoạn khác chứa?
- **Ràng buộc:** $1 \leq n \leq 2 \cdot 10^5, 1 \leq L_i < R_i \leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Vẫn sử dụng thứ tự sắp xếp: L tăng dần, R giảm dần.
- *Hint 2:* Nén các giá trị tọa độ R_i về tập rời rạc $\{1, 2, \dots, K\}$ với $K \leq n$.
- *Hint 3:* Sử dụng Cây Fenwick (Binary Indexed Tree):
 - Khi quét từ phải sang trái, truy vấn số phần tử trong BIT có tọa độ $\leq R_i$ để biết có bao nhiêu đoạn nằm trong nó, sau đó chèn R_i vào BIT.
 - Khi quét từ trái sang phải, truy vấn số phần tử trong BIT có tọa độ $\geq R_i$ để biết có bao nhiêu đoạn chứa nó. Độ phức tạp $O(n \log n)$.

Bài 7: Meeting Rooms II (LeetCode 253)

- **Đề bài:** Cho một mảng các khoảng thời gian hợp `intervals` với $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$. Tìm số lượng phòng họp tối thiểu cần chuẩn bị để mọi cuộc họp đều diễn ra đúng giờ mà không bị trùng phòng.
- **Ràng buộc:** $1 \leq n \leq 10^5, 0 \leq \text{start}_i < \text{end}_i \leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Sắp xếp các cuộc họp theo thời điểm bắt đầu start_i tăng dần.
- *Hint 2:* Sử dụng một Min-Heap để lưu trữ thời điểm kết thúc end của các cuộc họp đang sử dụng phòng.
- *Hint 3:* Khi cuộc họp tiếp theo bắt đầu tại start : nếu `heap.top() <= start`, cuộc họp trước đã kết thúc và phòng được giải phóng $\implies \text{heap.pop()}$. Đẩy end của cuộc họp mới vào heap. Kích thước lớn nhất của heap chính là số phòng cần thiết.

Bài 8: Maximum Profit in Job Scheduling (LeetCode 1235)

- **Đề bài:** Bạn có n công việc. Công việc i bắt đầu tại $\text{startTime}[i]$, kết thúc tại $\text{endTime}[i]$ và đem lại lợi nhuận $\text{profit}[i]$. Không thể làm 2 công việc trùng giờ (nhưng có thể làm công việc mới ngay tại thời điểm công việc cũ kết thúc). Hãy tìm lợi nhuận tối đa có thể kiếm được.
- **Ràng buộc:** $1 \leq n \leq 5 \cdot 10^4, 1 \leq \text{startTime}[i] < \text{endTime}[i] \leq 10^9, 1 \leq \text{profit}[i] \leq 10^4$.
- **Gợi ý:**
- *Hint 1:* Đây là bài toán Weighted Interval Scheduling. Sắp xếp các công việc theo thời điểm kết thúc endTime tăng dần.
- *Hint 2:* Gọi $\text{dp}[i]$ là lợi nhuận lớn nhất khi xét i công việc đầu tiên.
- *Hint 3:* Với công việc thứ i :
 - Không chọn: $\text{dp}[i - 1]$.

- Chọn: $profit[i] + dp[p]$, với p là công việc kết thúc $\leq startTime[i]$. Tìm p bằng `upper_bound` trên mảng `endTime`. Độ phức tạp: $O(n \log n)$.

Bài 9: Movie Festival II (CSES 1632)

- **Đề bài:** Có n bộ phim và k thành viên trong một câu lạc bộ phim. Mỗi thành viên có thể xem độc lập các bộ phim. Hãy tìm tổng số bộ phim nhiều nhất mà cả nhóm k người có thể xem (mỗi phim chỉ cần 1 người xem).
- **Ràng buộc:** $1 \leq k \leq n \leq 2 \cdot 10^5$, $1 \leq a_i < b_i \leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Sắp xếp các bộ phim theo thời điểm kết thúc b_i tăng dần.
- *Hint 2:* Duy trì thời điểm rảnh rỗi của cả k thành viên trong một `std::multiset`. Ban đầu chèn k số 0 vào multiset.
- *Hint 3:* Với mỗi phim $[a_i, b_i]$, ta tìm thành viên có thời điểm rảnh rỗi $\leq a_i$ và **lớn nhất có thể** bằng `upper_bound(a_i)` rồi lùi lại một vị trí (`prev`). Nếu tìm được, xóa mốc cũ và chèn b_i vào multiset.

Bài 10: Two Platforms (Codeforces 1409E)

- **Đề bài:** Có n điểm rơi trên trục số với tọa độ $x_1, \dots, x_n$. Bạn được đặt 2 tấm ván có cùng độ dài k . Hãy tìm số điểm tối đa mà 2 tấm ván có thể hứng được (hai tấm ván không được giao nhau nhưng có thể tiếp xúc).
- **Ràng buộc:** $1 \leq n \leq 2 \cdot 10^5$, $1 \leq k \leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Sắp xếp các tọa độ x tăng dần.
- *Hint 2:* Với mỗi điểm i , dùng hai con trỏ hoặc `upper_bound` để tính số điểm mà một tấm ván bắt đầu tại x_i phủ được: $L[i]$.
- *Hint 3:* Xây dựng mảng hậu tố cực đại: $R_max[i] = \max_{j \geq i} L[j]$. Khi tấm ván 1 phủ từ i đến r , tấm ván 2 tham lam lấy $R_max[r + 1]$.

Bài 11: Video Stitching (LeetCode 1024)

- **Đề bài:** Bạn có một danh sách các đoạn clip video $clips[i] = [start_i, end_i]$ và một khoảng thời gian mục tiêu $[0, time]$. Bạn muốn ghép các đoạn clip lại để tạo thành một video hoàn chỉnh dài từ 0 đến $time$. Tìm số lượng clip tối thiểu cần dùng, hoặc trả về -1 nếu không thể.
- **Ràng buộc:** $1 \leq clips.length \leq 100$, $0 \leq start_i < end_i \leq 100$, $1 \leq time \leq 100$.
- **Gợi ý:**
- *Hint 1:* Áp dụng thuật toán Tham lam Phủ đoạn tối thiểu (Minimum Interval Cover).
- *Hint 2:* Khởi tạo vị trí đã phủ `cur_end = 0`.
- *Hint 3:* Lặp lại: Trong các đoạn có $start \leq cur_end$, chọn đoạn có `end` lớn nhất. Cập nhật `cur_end = max(end)` và tăng biến đếm số clip. Nếu không thể đi tiếp trước khi đạt `time`, trả về -1 .

● Phần 3: Cấp độ Nâng cao & Thử thách Olympic (Advanced)

Bài 12: Packing Under Range Regulations (AtCoder ABC 214E)

- **Đề bài:** Có N quả bóng. Quả bóng thứ i chỉ được phép đặt vào một ô số nguyên thuộc đoạn $[L_i, R_i]$. Mỗi ô nguyên chỉ được chứa tối đa 1 quả bóng. Kiểm tra xem có tồn tại phương án đặt hợp lệ cho toàn bộ N quả bóng hay không.
- **Ràng buộc:** $1 \leq N \leq 2 \cdot 10^5, 1 \leq L_i \leq R_i \leq 10^9$.
- **Gợi ý:**
- *Hint 1:* Sắp xếp các quả bóng theo L_i tăng dần.
- *Hint 2:* Duy trì tọa độ hiện tại x và một Min-Heap lưu các hạn chót R của những quả bóng có $L_i \leq x$.
- *Hint 3:* Tại mỗi bước, lấy quả bóng có hạn chót R nhỏ nhất ra gán cho vị trí x . Nếu $R < x$, vi phạm ràng buộc $\implies$ No. Tăng $x \leftarrow x + 1$. Nếu heap rỗng, nhảy cóc $x = L$ của quả bóng tiếp theo để tránh TLE.

Bài 13: Moving Points (Codeforces 1311F)

- **Đề bài:** Có n điểm trên trục số thực. Điểm thứ i ban đầu ở x_i và di chuyển với vận tốc không đổi v_i . Với mỗi cặp điểm (i, j) ($i < j$), gọi $d(i, j)$ là khoảng cách nhỏ nhất giữa chúng trong tương lai ($t \geq 0$). Tính $\sum_{1 \leq i < j \leq n} d(i, j)$.
- **Ràng buộc:** $1 \leq n \leq 2 \cdot 10^5, -10^8 \leq x_i, v_i \leq 10^8$.
- **Gợi ý:**
- *Hint 1:* Giả sử $x_i \leq x_j$. Nếu $v_i \leq v_j$, hai điểm ngày càng rời xa $\implies d(i, j) = x_j - x_i$ tại $t = 0$. Nếu $v_i > v_j$, điểm i đuổi kịp điểm $j \implies d(i, j) = 0$.
- *Hint 2:* Bài toán quy về tính tổng $(x_j - x_i)$ trên tất cả các cặp thỏa mãn $x_i \leq x_j$ và $v_i \leq v_j$.
- *Hint 3:* Sắp xếp theo x tăng dần. Nén tọa độ vận tốc v và sử dụng 2 cây Fenwick: một cây đếm số lượng điểm có vận tốc $\leq v_j$, một cây tính tổng tọa độ x_i .

Bài 14: Remove Covered Intervals (LeetCode 1288)

- **Đề bài:** Cho một danh sách các đoạn thẳng. Hãy xóa tất cả các đoạn thẳng bị bao trọn bởi một đoạn thẳng khác trong danh sách. Trả về số lượng đoạn thẳng còn lại.
- **Ràng buộc:** $1 \leq n \leq 1000, 0 \leq L_i < R_i \leq 10^5$.
- **Gợi ý:**
- *Hint 1:* Sắp xếp theo L tăng dần; nếu L bằng nhau thì R giảm dần.
- *Hint 2:* Duy trì `max_r = 0`. Với mỗi đoạn $[l, r]$: nếu $r \leq \text{max_r}$, đoạn này chắc chắn bị bao bởi một đoạn trước đó $\implies$ bỏ qua.
- *Hint 3:* Nếu $r > \text{max_r}$, đoạn này không bị bao; ta giữ lại đoạn này và cập nhật $\text{max_r} = r$.

Bài 15: Interval Selection (AtCoder ARC 067D / Olympic)

- **Đề bài:** Cho N đoạn thẳng $[L_i, R_i]$. Cần chọn ra một tập con các đoạn thẳng sao cho tại mọi điểm trên trục số, không có điểm nào bị phủ bởi quá 2 đoạn thẳng, đồng thời tối đa hóa số đoạn thẳng được chọn.
- **Ràng buộc:** $1 \leq N \leq 10^5$.
- **Gợi ý:**
 - *Hint 1:* Đây là bài toán mở rộng của Interval Scheduling với sức chứa (Capacity) bằng 2.
 - *Hint 2:* Sắp xếp các đoạn theo R_i tăng dần.
 - *Hint 3:* Dùng Segment Tree với Lazy Propagation quản lý số tầng phủ trên trục tọa độ đã nén. Khi xét một đoạn $[L_i, R_i]$, truy vấn $\max$ trên đoạn $[L_i, R_i]$: nếu $\max < 2$, ta có thể chọn đoạn này và cập nhật $+1$ trên đoạn $[L_i, R_i]$.

Bài 16: Maximum White Subtree (Codeforces 1324F)

- **Đề bài:** Cho một cây n đỉnh với màu trắng hoặc đen. Với mỗi đỉnh u , tìm một cây con liên thông chứa u sao cho hiệu (số đỉnh trắng — số đỉnh đen) là lớn nhất.
- **Ràng buộc:** $1 \leq n \leq 2 \cdot 10^5$.
- **Gợi ý:**
 - *Hint 1:* Biến đổi gán giá trị: Đỉnh trắng mang trọng số $+1$, đỉnh đen mang trọng số -1 .
 - *Hint 2:* Áp dụng kỹ thuật Quy hoạch động đổi gốc (Rerooting DP / Tree DP) 2 lượt DFS.
 - *Hint 3:* Lượt DFS 1 tính $dp[u]$ cho cây con gốc u : $dp[u] = val[u] + \sum_{v \in child(u)} \max(0, dp[v])$. Lượt DFS 2 đẩy kết quả từ nút cha xuống nút con trong $O(1)$.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.